

Afficionado Coffee Roasters

Import File

```
from google.colab import files
```

```
uploaded = files.upload()
```

[Choose Files](#) Afficionado ...ers.xlsx.csv

Afficionado Coffee Roasters.xlsx.csv(text/csv) - 12925448 bytes, last modified: 26/6/2026 - 100% done

Saving Afficionado Coffee Roasters.xlsx.csv to Afficionado Coffee Roasters.xlsx.csv

Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Load Data

```
df = pd.read_csv("Afficionado Coffee Roasters.xlsx.csv")
```

Check Data

```
df.head(), df.shape, df.info(), df.columns
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 149116 entries, 0 to 149115
Data columns (total 11 columns):
 #   Column                Non-Null Count  Dtype  
---  -
 0   transaction_id         149116 non-null  int64  
 1   year                  149116 non-null  int64  
 2   transaction_time       149116 non-null  object  
 3   transaction_qty        149116 non-null  int64  
 4   store_id              149116 non-null  int64  
 5   store_location         149116 non-null  object  
 6   product_id            149116 non-null  int64  
 7   unit_price            149116 non-null  float64 
 8   product_category       149116 non-null  object  
 9   product_type           149116 non-null  object  
10   product_detail         149116 non-null  object  
dtypes: float64(1), int64(5), object(5)
memory usage: 12.5+ MB
(  transaction_id  year  transaction_time  transaction_qty  store_id \
0                1  2025          7:06:11                2         5
1                2  2025          7:08:56                2         5
2                3  2025          7:14:04                2         5
3                4  2025          7:20:24                1         5
4                5  2025          7:22:41                2         5

   store_location  product_id  unit_price  product_category \
0  Lower Manhattan        32         3.0             Coffee
1  Lower Manhattan        57         3.1              Tea
2  Lower Manhattan        59         4.5  Drinking Chocolate
3  Lower Manhattan        22         2.0             Coffee
4  Lower Manhattan        57         3.1              Tea

   product_type  product_detail
0  Gourmet brewed coffee      Ethiopia Rg
1    Brewed Chai tea  Spicy Eye Opener Chai Lg
2    Hot chocolate      Dark chocolate Lg
3    Drip coffee    Our Old Time Diner Blend Sm
4    Brewed Chai tea  Spicy Eye Opener Chai Lg ,
(149116, 11),
None,
Index(['transaction_id', 'year', 'transaction_time', 'transaction_qty',
      'store_id', 'store_location', 'product_id', 'unit_price',
      'product_category', 'product_type', 'product_detail'],
      dtype='object'))
```

Data Cleaning

Missing Values

```
df.isnull().sum()
```

	0
transaction_id	0
year	0
transaction_time	0
transaction_qty	0
store_id	0
store_location	0
product_id	0
unit_price	0
product_category	0
product_type	0
product_detail	0

dtype: int64

```
df["product_id"].isnull().sum()
```

```
np.int64(0)
```

Duplicate Records

```
df.duplicated().sum()
```

```
np.int64(0)
```

```
df = df.drop_duplicates()
```

```
df.groupby("product_id")["product_detail"].nunique().sort_values(ascending=False)
```

product_detail	
product_id	
1	1
2	1
3	1
4	1
5	1
...	...
81	1
82	1
83	1
84	1
87	1

80 rows × 1 columns

dtype: int64

Validate Quantities

```
df[df["transaction_qty"] <= 0]
```

transaction_id	year	transaction_time	transaction_qty	store_id	store_location	product_id	unit_price	product_category
----------------	------	------------------	-----------------	----------	----------------	------------	------------	------------------

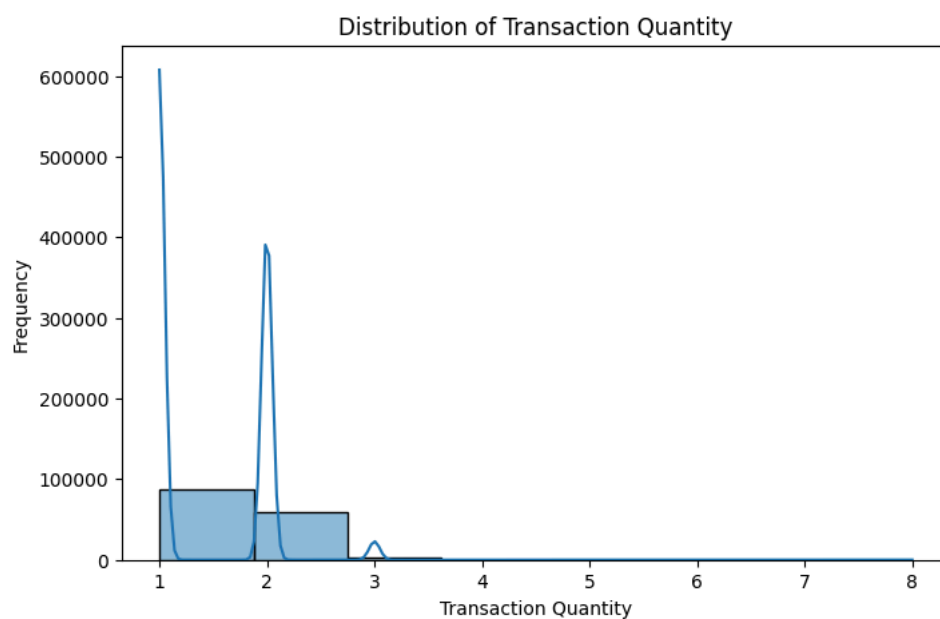
```
df = df[df["transaction_qty"] > 0]
```

```
df["transaction_qty"].describe()
```

	transaction_qty
count	149116.000000
mean	1.438276
std	0.542509
min	1.000000
25%	1.000000
50%	1.000000
75%	2.000000
max	8.000000

dtype: float64

```
plt.figure(figsize=(8,5))
sns.histplot(df["transaction_qty"], bins=8, kde=True)
plt.title("Distribution of Transaction Quantity")
plt.xlabel("Transaction Quantity")
plt.ylabel("Frequency")
plt.show()
```



Validate Unit Price

```
df[df["unit_price"] <= 0]
```

transaction_id	year	transaction_time	transaction_qty	store_id	store_location	product_id	unit_price	product_category
----------------	------	------------------	-----------------	----------	----------------	------------	------------	------------------

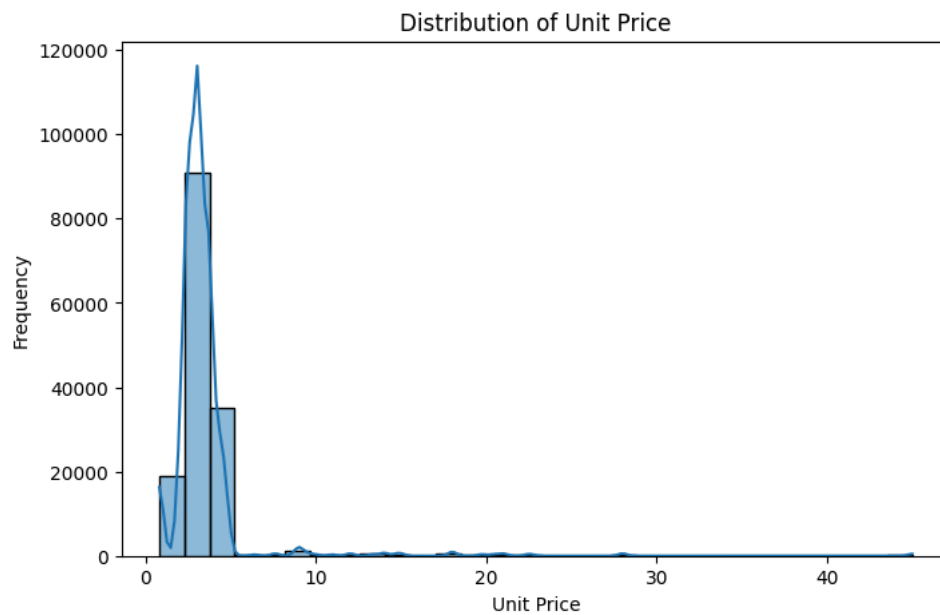
```
df = df[df["unit_price"] > 0]
```

```
df["unit_price"].describe()
```

	unit_price
count	149116.000000
mean	3.382219
std	2.658723
min	0.800000
25%	2.500000
50%	3.000000
75%	3.750000
max	45.000000

dtype: float64

```
plt.figure(figsize=(8,5))
sns.histplot(df["unit_price"], bins=30, kde=True)
plt.title("Distribution of Unit Price")
plt.xlabel("Unit Price")
plt.ylabel("Frequency")
plt.show()
```



Create Revenue

```
df["revenue"] = (df["transaction_qty"] * df["unit_price"])
```

```
df[["transaction_qty", "unit_price", "revenue"]].head()
```

	transaction_qty	unit_price	revenue	
0	2	3.0	6.0	
1	2	3.1	6.2	
2	2	4.5	9.0	
3	1	2.0	2.0	
4	2	3.1	6.2	

Exporting Cleaned Data

```
df.to_csv("cleaned_data.csv", index=False)
```

```
files.download("cleaned_data.csv")
```

Revenue Computation

Revenue by Product

```
product_revenue = (
    df.groupby("product_detail")
      .agg(
          total_revenue=("revenue", "sum")
      )
      .reset_index()
)

product_revenue.head()
```

	product_detail	total_revenue	
0	Almond Croissant	7168.13	
1	Brazilian - Organic	3852.00	
2	Brazilian Lg	15109.50	
3	Brazilian Rg	13155.00	
4	Brazilian Sm	9482.00	

Next steps:

[Generate code with product_revenue](#)

[New interactive sheet](#)

Revenue by Product Type

```
type_revenue = (
    df.groupby("product_type")
      .agg(
        total_revenue=("revenue", "sum")
      )
      .reset_index()
)

type_revenue
```

	product_type	total_revenue	
0	Barista Espresso	91406.20	
1	Biscotti	19793.53	
2	Black tea	2711.85	
3	Brewed Black tea	47932.00	
4	Brewed Chai tea	77081.95	
5	Brewed Green tea	23852.50	
6	Brewed herbal tea	47539.50	
7	Chai tea	4301.25	
8	Clothing	6163.00	
9	Drinking Chocolate	2728.04	
10	Drip coffee	31984.00	
11	Espresso Beans	5560.25	
12	Gourmet Beans	6798.00	
13	Gourmet brewed coffee	70034.60	
14	Green beans	1340.00	
15	Green tea	1470.75	
16	Herbal tea	2729.75	
17	Hot chocolate	72416.00	
18	House blend Beans	3294.00	
19	Housewares	7444.00	
20	Organic Beans	8509.50	
21	Organic Chocolate	1679.60	
22	Organic brewed coffee	37746.50	
23	Pastry	25655.99	
24	Premium Beans	14583.50	
25	Premium brewed coffee	38781.15	
26	Regular syrup	6084.80	
27	Scone	36866.12	
28	Sugar free syrup	2324.00	

Next steps:

[Generate code with type_revenue](#)

[New interactive sheet](#)

```
top15 = type_revenue.sort_values(
    "total_revenue"
```

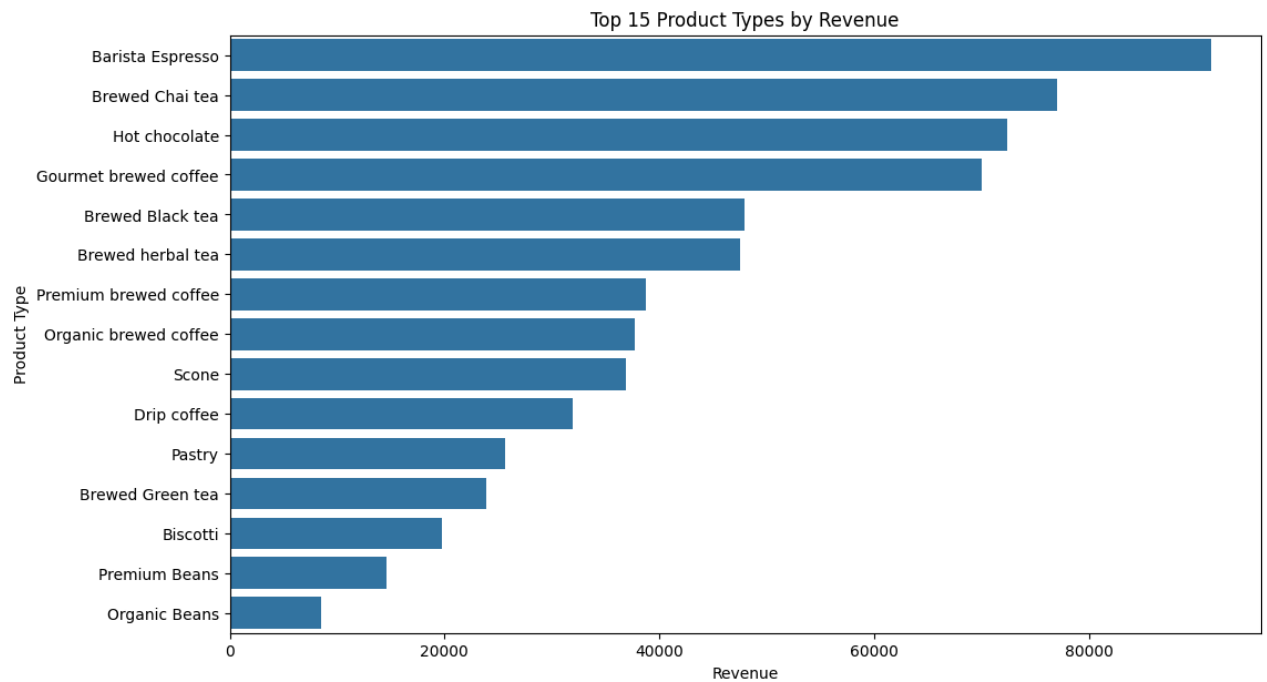
```

        ascending=False
    ).head(15)

plt.figure(figsize=(12,7))
sns.barplot(
    data=top15,
    x="total_revenue",
    y="product_type"
)

plt.title("Top 15 Product Types by Revenue")
plt.xlabel("Revenue")
plt.ylabel("Product Type")
plt.show()

```



Revenue by Category

```

category_revenue = (
    df.groupby("product_category")
      .agg(
          total_revenue=("revenue", "sum")
      )
      .reset_index()
)

category_revenue

```

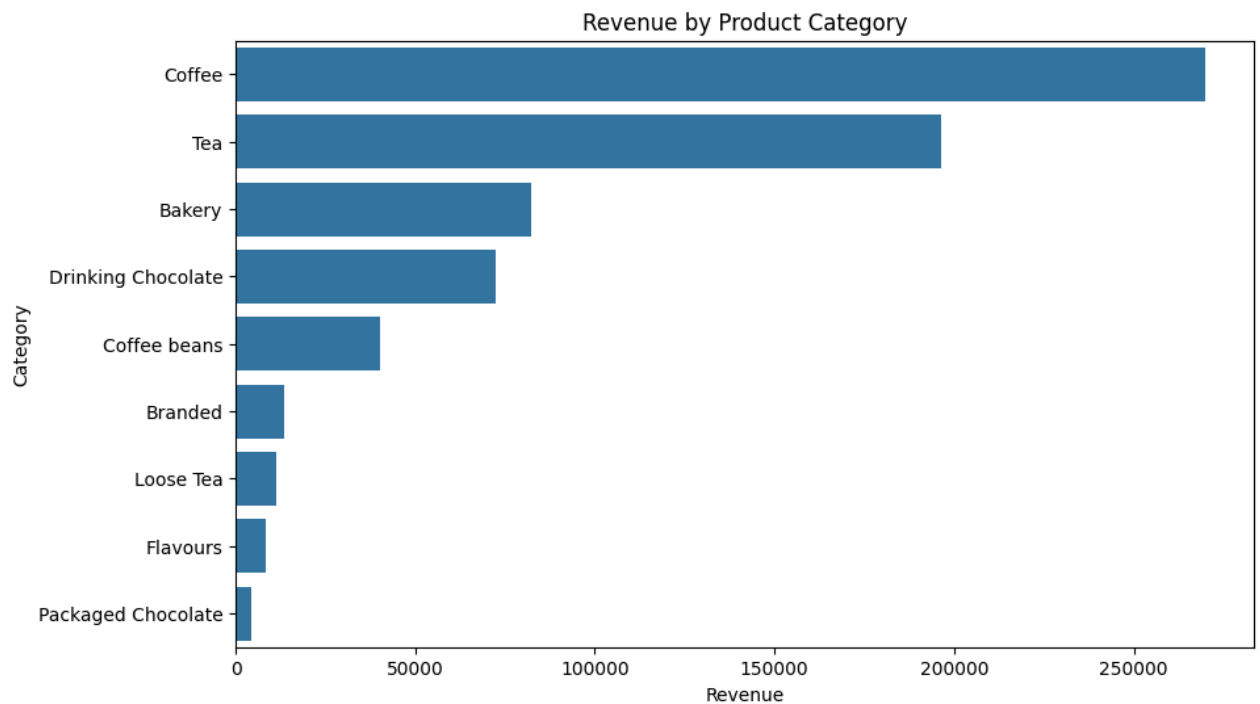
	product_category	total_revenue	
0	Bakery	82315.64	
1	Branded	13607.00	
2	Coffee	269952.45	
3	Coffee beans	40085.25	
4	Drinking Chocolate	72416.00	
5	Flavours	8408.80	
6	Loose Tea	11213.60	
7	Packaged Chocolate	4407.64	
8	Tea	196405.95	

Next steps: [Generate code with category_revenue](#) [New interactive sheet](#)

```
category_revenue_df = category_revenue.reset_index()
category_revenue_df = category_revenue_df.rename(columns={'revenue': 'total_revenue'})

plt.figure(figsize=(10,6))
sns.barplot(
    data=category_revenue_df.sort_values("total_revenue", ascending=False),
    x="total_revenue",
    y="product_category"
)

plt.title("Revenue by Product Category")
plt.xlabel("Revenue")
plt.ylabel("Category")
plt.show()
```



Product Popularity Analysis

Total Units Sold Per Product

```
product_volume = (
    df.groupby("product_detail")
      .agg(
        units_sold=("transaction_qty", "sum")
      )
      .reset_index()
)

product_volume.head()
```

	product_detail	units_sold
0	Almond Croissant	1911
1	Brazilian - Organic	214
2	Brazilian Lg	4317
3	Brazilian Rg	4385
4	Brazilian Sm	4310

Next steps: [Generate code with product_volume](#) [New interactive sheet](#)

Ranking Products

```
product_volume["volume_rank"] = (
    product_volume["units_sold"]
      .rank(
        ascending=False,
```

```
        method="dense"
    )
)
```

Top 10 Products

```
top10_volume = (
    product_volume
    .sort_values(
        "units_sold",
        ascending=False
    )
    .head(10)
)

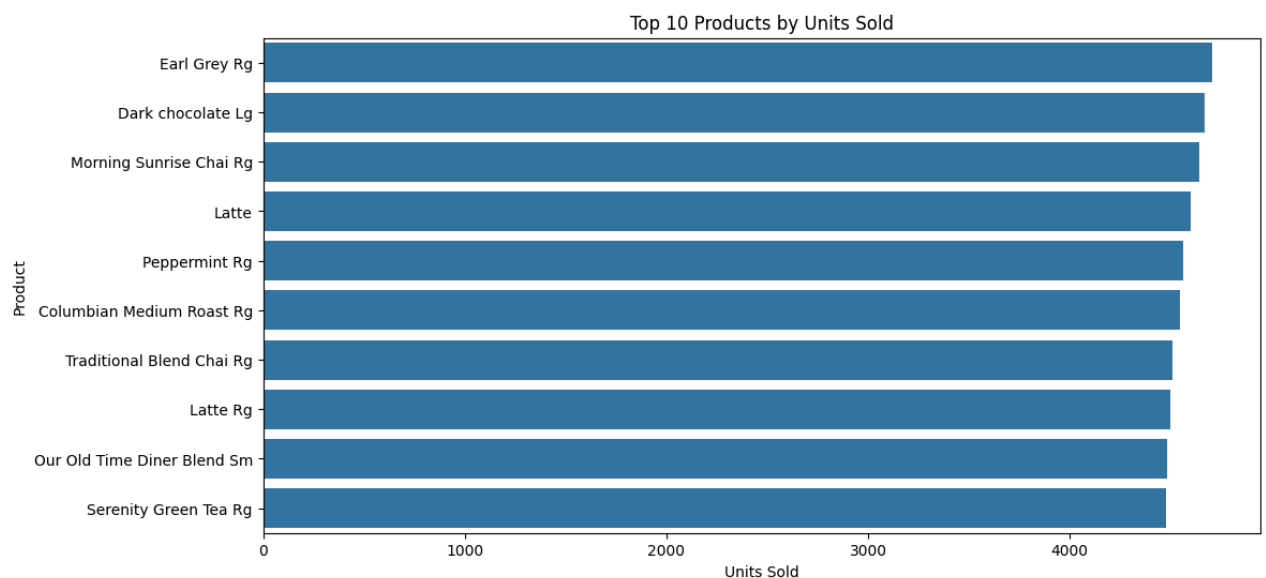
top10_volume
```

	product_detail	units_sold	volume_rank	
24	Earl Grey Rg	4708	1.0	
20	Dark chocolate Lg	4668	2.0	
54	Morning Sunrise Chai Rg	4643	3.0	
47	Latte	4602	4.0	
64	Peppermint Rg	4564	5.0	
15	Columbian Medium Roast Rg	4547	6.0	
79	Traditional Blend Chai Rg	4512	7.0	
48	Latte Rg	4497	8.0	
60	Our Old Time Diner Blend Sm	4484	9.0	
69	Serenity Green Tea Rg	4477	10.0	

Next steps: [Generate code with top10_volume](#) [New interactive sheet](#)

```
plt.figure(figsize=(12,6))
sns.barplot(
    data=top10_volume,
    x="units_sold",
    y="product_detail"
)

plt.title("Top 10 Products by Units Sold")
plt.xlabel("Units Sold")
plt.ylabel("Product")
plt.show()
```



Bottom 10 Products

```
bottom10_volume = (
    product_volume
    .sort_values(
        "units_sold",
        ascending=True
    )
    .head(10)
)

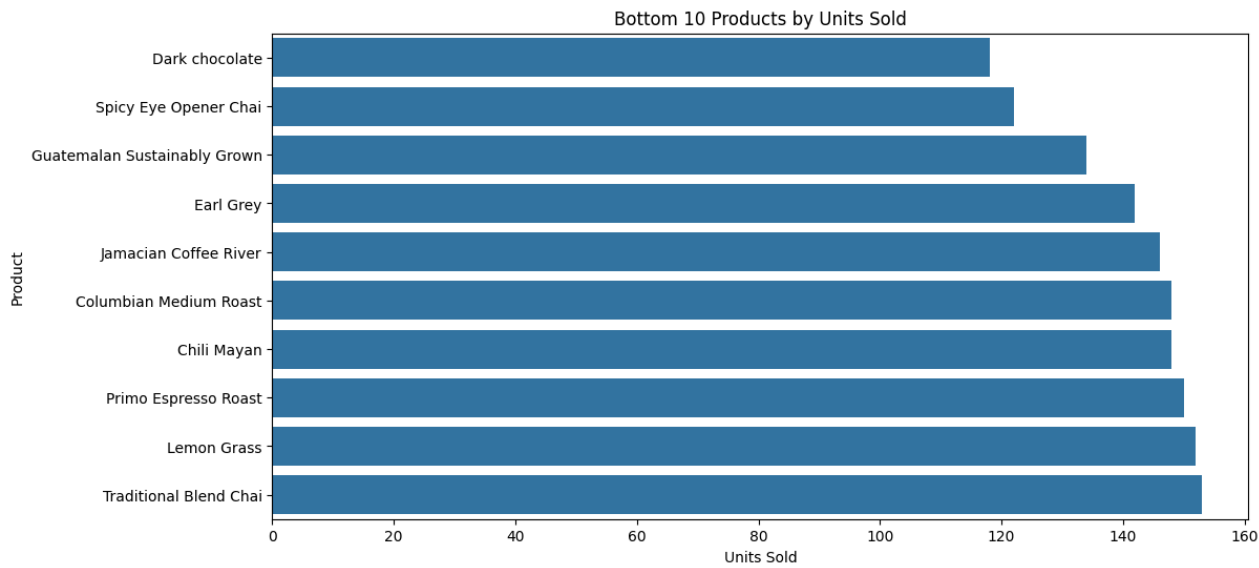
bottom10_volume
```

	product_detail	units_sold	volume_rank	
19	Dark chocolate	118	75.0	
70	Spicy Eye Opener Chai	122	74.0	
36	Guatemalan Sustainably Grown	134	73.0	
22	Earl Grey	142	72.0	
42	Jamacian Coffee River	146	71.0	
13	Columbian Medium Roast	148	70.0	
8	Chili Mayan	148	70.0	
65	Primo Espresso Roast	150	69.0	
49	Lemon Grass	152	68.0	
77	Traditional Blend Chai	153	67.0	

Next steps: [Generate code with bottom10_volume](#) [New interactive sheet](#)

```
plt.figure(figsize=(12,6))
sns.barplot(
    data=bottom10_volume,
    x="units_sold",
    y="product_detail"
)

plt.title("Bottom 10 Products by Units Sold")
plt.xlabel("Units Sold")
plt.ylabel("Product")
plt.show()
```



Revenue Contribution Analysis

Revenue Per Product

```
revenue_per_product = df['transaction_qty'].sum()/df['unit_price'].sum()
revenue_per_product
```

```
np.float64(0.42524628525152813)
```

Revenue Share %

```
total_revenue = df["revenue"].sum()

product_revenue["revenue_share_pct"] = (
    product_revenue["total_revenue"]
    / total_revenue
) * 100
total_revenue
```

```
np.float64(698812.3300000002)
```

Revenue Ranking

```
product_revenue["revenue_rank"] = (
    product_revenue["total_revenue"]
    .rank(
        ascending=False,
        method="dense"
    )
)
product_revenue.head()
```

	product_detail	total_revenue	revenue_share_pct	revenue_rank	
0	Almond Croissant	7168.13	1.025759	47.0	
1	Brazilian - Organic	3852.00	0.551221	58.0	
2	Brazilian Lg	15109.50	2.162168	10.0	
3	Brazilian Rg	13155.00	1.882480	15.0	
4	Brazilian Sm	9482.00	1.356874	40.0	

Next steps: [Generate code with product_revenue](#) [New interactive sheet](#)

Compare Volume Rank vs Revenue Rank

```
comparison = (
    product_volume.merge(
        product_revenue,
        on="product_detail"
    )
)
comparison.head()
```

	product_detail	units_sold	volume_rank	total_revenue	revenue_share_pct	revenue_rank	
0	Almond Croissant	1911	52.0	7168.13	1.025759	47.0	
1	Brazilian - Organic	214	60.0	3852.00	0.551221	58.0	
2	Brazilian Lg	4317	22.0	15109.50	2.162168	10.0	
3	Brazilian Rg	4385	17.0	13155.00	1.882480	15.0	
4	Brazilian Sm	4310	23.0	9482.00	1.356874	40.0	

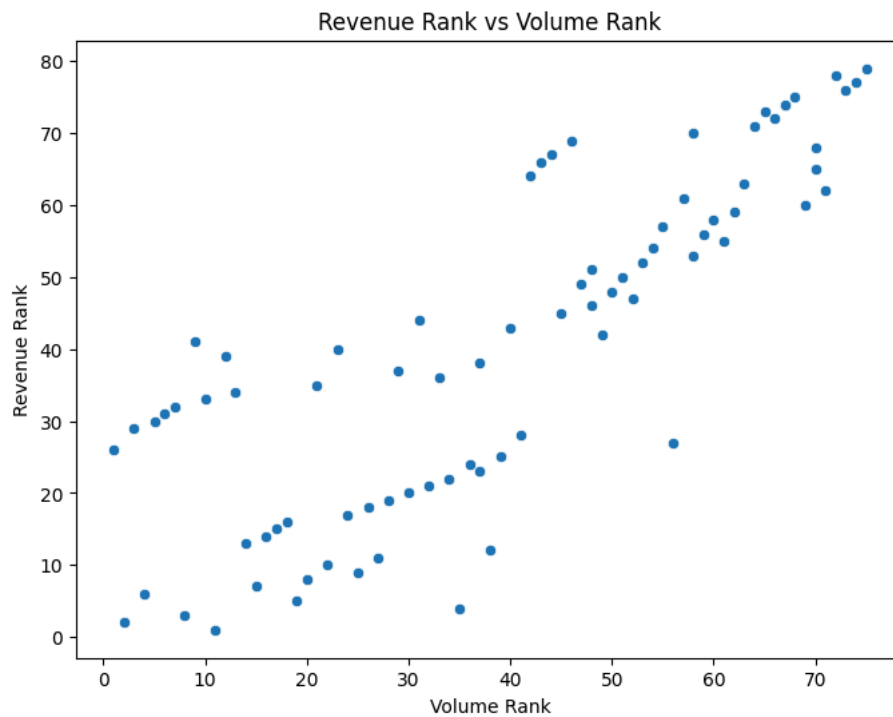
Next steps: [Generate code with comparison](#) [New interactive sheet](#)

```
plt.figure(figsize=(8,6))

sns.scatterplot(
    data=comparison,
    x="volume_rank",
    y="revenue_rank"
)

plt.title("Revenue Rank vs Volume Rank")
```

```
plt.xlabel("Volume Rank")
plt.ylabel("Revenue Rank")
plt.show()
```



Rank Gap

```
comparison["rank_gap"] = (
    comparison["volume_rank"]
    -
    comparison["revenue_rank"]
)
comparison["rank_gap"].head()
```

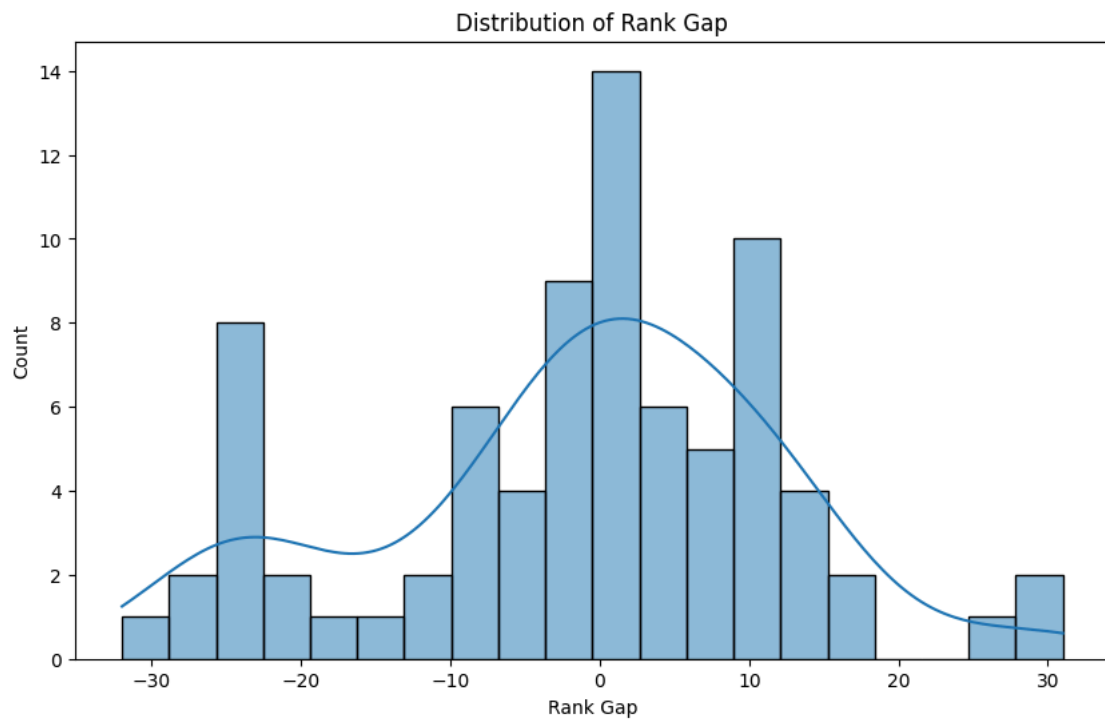
	rank_gap
0	5.0
1	2.0
2	12.0
3	2.0
4	-17.0

dtype: float64

```
plt.figure(figsize=(10,6))

sns.histplot(
    comparison["rank_gap"],
    bins=20,
    kde=True
)

plt.title("Distribution of Rank Gap")
plt.xlabel("Rank Gap")
plt.ylabel("Count")
plt.show()
```



Category & Product-Type Performance

Revenue Share by Category

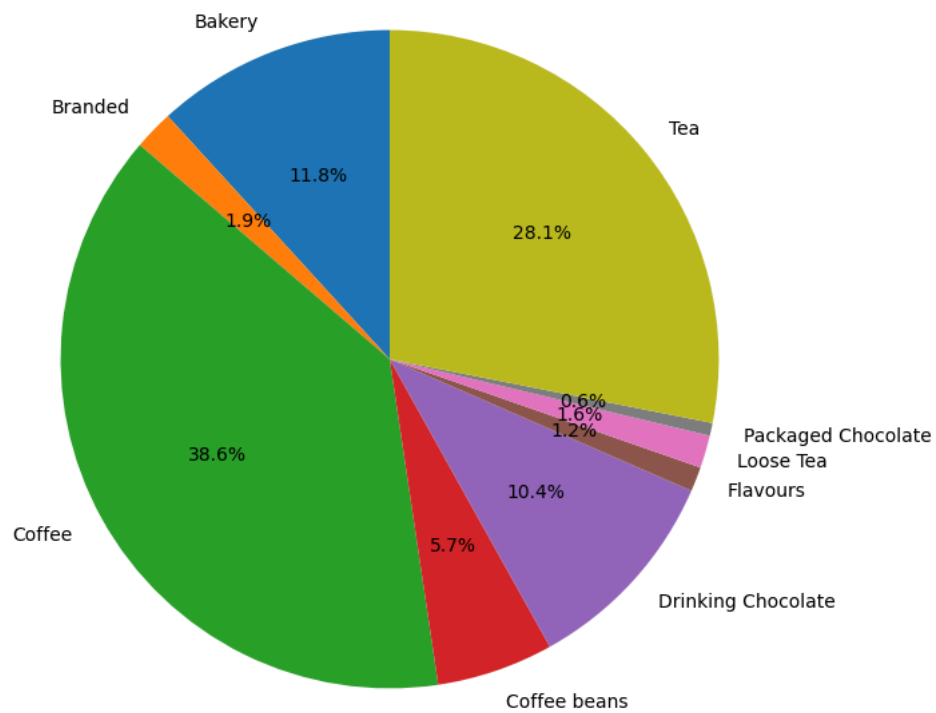
```
category_revenue["category_share_pct"] = (  
    category_revenue["total_revenue"]  
    /  
    category_revenue["total_revenue"].sum()  
    ) * 100  
category_revenue["category_share_pct"]
```

	category_share_pct
0	11.779363
1	1.947161
2	38.630178
3	5.736197
4	10.362725
5	1.203299
6	1.604665
7	0.630733
8	28.105679

dtype: float64

```
plt.figure(figsize=(8,8))  
  
plt.pie(  
    category_revenue_df["total_revenue"],  
    labels=category_revenue_df["product_category"],  
    autopct="%1.1f%%",  
    startangle=90  
)  
  
plt.title("Revenue Share by Category")  
plt.show()
```

Revenue Share by Category



Product-Type Contribution

```
type_revenue["type_share_pct"] = (  
    type_revenue["total_revenue"]  
    /  
    type_revenue["total_revenue"].sum()  
    ) * 100  
type_revenue["type_share_pct"]
```

	type_share_pct
0	13.080221
1	2.832453
2	0.388066
3	6.859066
4	11.030422
5	3.413291
6	6.802899
7	0.615509
8	0.881925
9	0.390382
10	4.576908
11	0.795671
12	0.972793
13	10.021947
14	0.191754
15	0.210464
16	0.390627
17	10.362725
18	0.471371
19	1.065236
20	1.217709
21	0.240351
22	5.401522
23	3.671371
24	2.086898
25	5.549580
26	0.870734
27	5.275539
28	0.332564

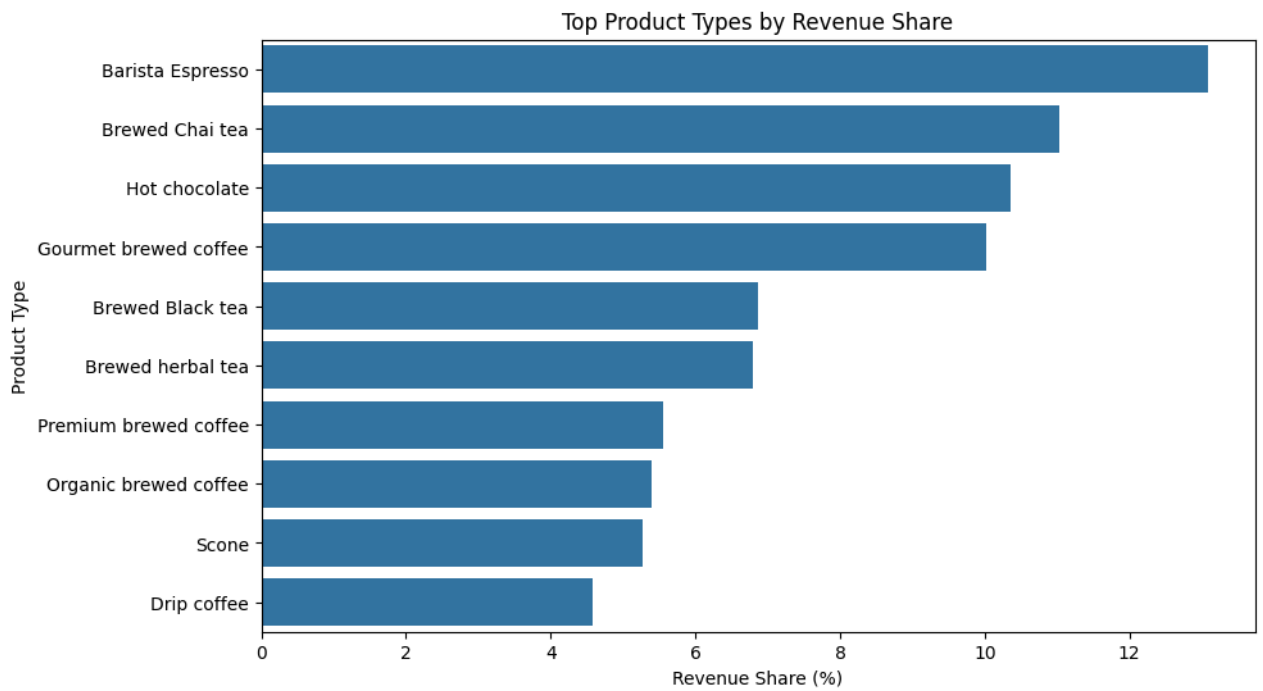
dtype: float64

```
top10 = type_revenue.sort_values(
    "type_share_pct",
    ascending=False
).head(10)

plt.figure(figsize=(10,6))

sns.barplot(
    data=top10,
    x="type_share_pct",
    y="product_type"
)

plt.title("Top Product Types by Revenue Share")
plt.xlabel("Revenue Share (%)")
plt.ylabel("Product Type")
plt.show()
```



Core Category Dependence

```
category_revenue.sort_values(  
    "total_revenue",  
    ascending=False  
)  
category_revenue
```

	product_category	total_revenue	category_share_pct	
0	Bakery	82315.64	11.779363	
1	Branded	13607.00	1.947161	
2	Coffee	269952.45	38.630178	
3	Coffee beans	40085.25	5.736197	
4	Drinking Chocolate	72416.00	10.362725	
5	Flavours	8408.80	1.203299	
6	Loose Tea	11213.60	1.604665	
7	Packaged Chocolate	4407.64	0.630733	
8	Tea	196405.95	28.105679	

Next steps: [Generate code with category_revenue](#) [New interactive sheet](#)

Revenue Concentration & Menu Balance

Sort Products

```
pareto = (  
    product_revenue  
    .sort_values(  
        "total_revenue",  
        ascending=False  
    )  
    .reset_index(drop=True)  
)  
pareto
```

	product_detail	total_revenue	revenue_share_pct	revenue_rank	
0	Sustainably Grown Organic Lg	21151.75	3.026814	1.0	 
1	Dark chocolate Lg	21006.00	3.005957	2.0	
2	Latte Rg	19112.25	2.734962	3.0	
3	Cappuccino Lg	17641.75	2.524533	4.0	
4	Morning Sunrise Chai Lg	17384.00	2.487649	5.0	
...	...	...	...	...	
75	Lemon Grass	1360.40	0.194673	75.0	
76	Guatemalan Sustainably Grown	1340.00	0.191754	76.0	
77	Spicy Eye Opener Chai	1335.90	0.191167	77.0	
78	Earl Grey	1270.90	0.181866	78.0	
79	Dark chocolate	755.20	0.108069	79.0	

80 rows × 4 columns

Next steps:

[Generate code with pareto](#)

[New interactive sheet](#)

Cumulative Revenue %

```
pareto["cumulative_pct"] = (
    pareto["total_revenue"]
    .cumsum()
    /
    pareto["total_revenue"].sum()
) * 100
pareto
```

	product_detail	total_revenue	revenue_share_pct	revenue_rank	cumulative_pct	
0	Sustainably Grown Organic Lg	21151.75	3.026814	1.0	3.026814	 
1	Dark chocolate Lg	21006.00	3.005957	2.0	6.032771	
2	Latte Rg	19112.25	2.734962	3.0	8.767733	
3	Cappuccino Lg	17641.75	2.524533	4.0	11.292266	
4	Morning Sunrise Chai Lg	17384.00	2.487649	5.0	13.779916	
...	...	...	...	...	...	
75	Lemon Grass	1360.40	0.194673	75.0	99.327144	
76	Guatemalan Sustainably Grown	1340.00	0.191754	76.0	99.518898	
77	Spicy Eye Opener Chai	1335.90	0.191167	77.0	99.710065	
78	Earl Grey	1270.90	0.181866	78.0	99.891931	
79	Dark chocolate	755.20	0.108069	79.0	100.000000	

80 rows × 5 columns

Next steps:

[Generate code with pareto](#)

[New interactive sheet](#)

```
fig, ax1 = plt.subplots(figsize=(14,6))

sns.barplot(
    data=pareto,
    x="product_detail",
    y="total_revenue",
    color="skyblue",
    ax=ax1
)

ax1.tick_params(axis='x', rotation=90)
ax1.set_ylabel("Revenue")

ax2 = ax1.twinx()

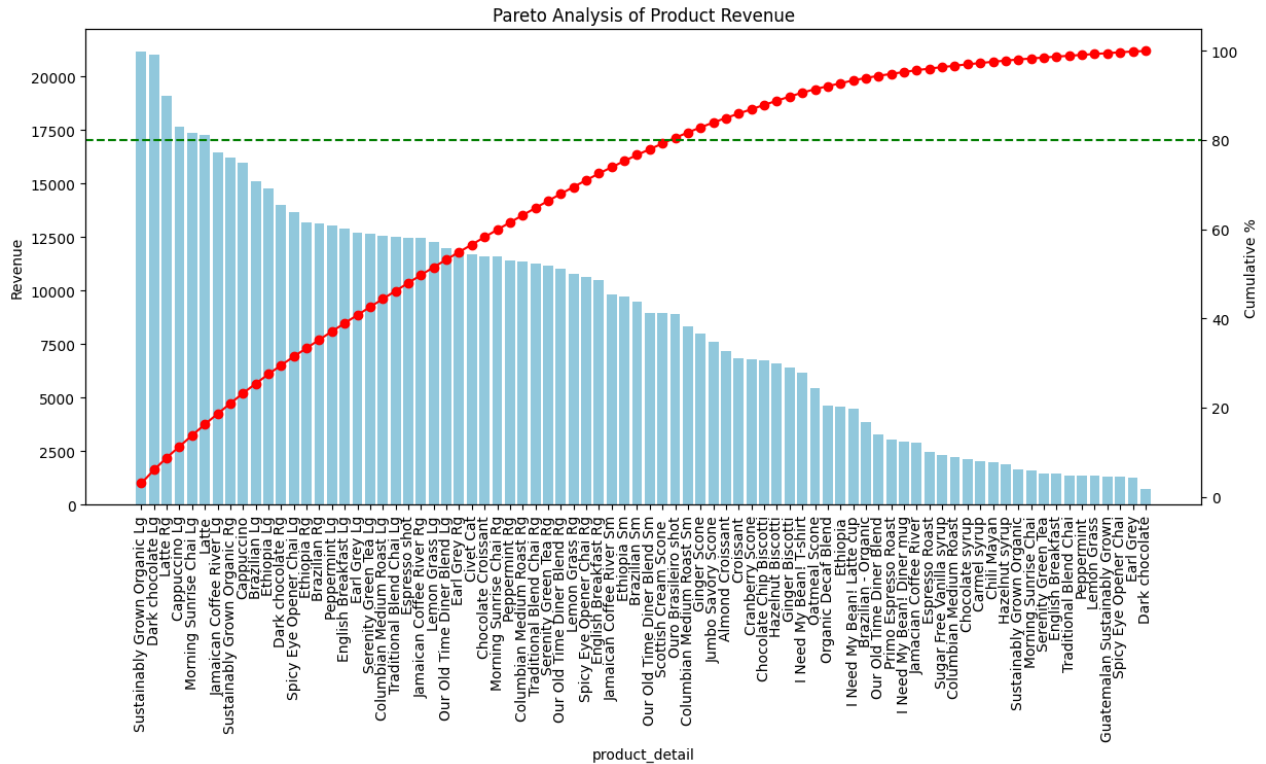
ax2.plot(
    pareto.index,
    pareto["cumulative_pct"],
    color="red",
    marker="o"
```



```
)

ax2.set_ylabel("Cumulative %")
ax2.axhline(80, color="green", linestyle="--")

plt.title("Pareto Analysis of Product Revenue")
plt.show()
```



Pareto Classification

```
pareto["category"] = np.where(
    pareto["cumulative_pct"] <= 80,
    "Revenue Anchor",
    "Long Tail"
)
pareto
```

	product_detail	total_revenue	revenue_share_pct	revenue_rank	cumulative_pct	category	
0	Sustainably Grown Organic Lg	21151.75	3.026814	1.0	3.026814	Revenue Anchor	
1	Dark chocolate Lg	21006.00	3.005957	2.0	6.032771	Revenue Anchor	
2	Latte Rg	19112.25	2.734962	3.0	8.767733	Revenue Anchor	
3	Cappuccino Lg	17641.75	2.524533	4.0	11.292266	Revenue Anchor	
4	Morning Sunrise Chai Lg	17384.00	2.487649	5.0	13.779916	Revenue Anchor	
...	...	...	...	...	...	...	
75	Lemon Grass	1360.40	0.194673	75.0	99.327144	Long Tail	
76	Guatemalan Sustainably Grown	1340.00	0.191754	76.0	99.518898	Long Tail	
77	Spicy Eye Opener Chai	1335.90	0.191167	77.0	99.710065	Long Tail	
78	Earl Grey	1270.90	0.181866	78.0	99.891931	Long Tail	
79	Dark chocolate	755.20	0.108069	79.0	100.000000	Long Tail	

80 rows × 6 columns

Next steps: [Generate code with pareto](#) [New interactive sheet](#)

Revenue Anchors

```
anchors = (  
    pareto[  
        pareto["category"]  
        == "Revenue Anchor"  
    ]  
)  
anchors.head()
```

	product_detail	total_revenue	revenue_share_pct	revenue_rank	cumulative_pct	category	
0	Sustainably Grown Organic Lg	21151.75	3.026814	1.0	3.026814	Revenue Anchor	
1	Dark chocolate Lg	21006.00	3.005957	2.0	6.032771	Revenue Anchor	
2	Latte Rg	19112.25	2.734962	3.0	8.767733	Revenue Anchor	
3	Cappuccino Lg	17641.75	2.524533	4.0	11.292266	Revenue Anchor	
4	Morning Sunrise Chai Lg	17384.00	2.487649	5.0	13.779916	Revenue Anchor	

Next steps: [Generate code with anchors](#) [New interactive sheet](#)

Start coding or [generate](#) with AI.

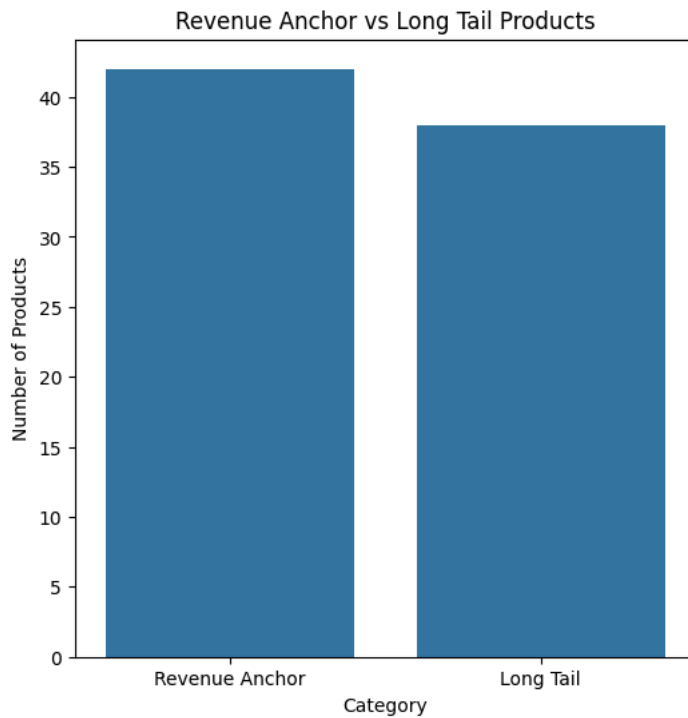
Long Tail Products

```
long_tail = (  
    pareto[  
        pareto["category"]  
        == "Long Tail"  
    ]  
)  
long_tail.head()
```

	product_detail	total_revenue	revenue_share_pct	revenue_rank	cumulative_pct	category	
42	Ouro Brasileiro shot	8902.20	1.273904	43.0	80.521451	Long Tail	
43	Columbian Medium Roast Sm	8356.00	1.195743	44.0	81.717194	Long Tail	
44	Ginger Scone	8011.61	1.146461	45.0	82.863655	Long Tail	
45	Jumbo Savory Scone	7626.62	1.091369	46.0	83.955024	Long Tail	
46	Almond Croissant	7168.13	1.025759	47.0	84.980783	Long Tail	

Next steps: [Generate code with long_tail](#) [New interactive sheet](#)

```
plt.figure(figsize=(6,6))  
  
sns.countplot(  
    data=pareto,  
    x="category"  
)  
  
plt.title("Revenue Anchor vs Long Tail Products")  
plt.xlabel("Category")  
plt.ylabel("Number of Products")  
plt.show()
```



Diversification Risk

```
len(anchors)
```

42

```
len(long_tail)
```

38

KPI Creation

Product Revenue Contribution %

```
total_revenue = df["revenue"].sum()
product_revenue = (df.groupby("product_detail")["revenue"].sum())

highest_contribution = (product_revenue.max() / total_revenue) * 100

print(round(highest_contribution, 2))
```

3.03

Product Sales Volume

```
total_units = df["transaction_qty"].sum()

print(total_units)
```

214470

Category Revenue Share

```
category_revenue = (
    df.groupby("product_category")["revenue"]
    .sum()
)

highest_category_share = (
    category_revenue.max() /
    category_revenue.sum()
) * 100

print(round(highest_category_share, 2))
```

38.63

Revenue Concentration Ratio

```
top_products = pareto[pareto["cumulative_pct"] <= 80]

ratio = (
    top_products["total_revenue"].sum()
    /
    total_revenue
) * 100

print(round(ratio,2))
```

79.25

Product Efficiency Score

```
efficiency = (
    df["revenue"].sum()
    /
    df["transaction_qty"].sum()
)

print(round(efficiency,2))
```

3.26

```
store_revenue = (
    df.groupby("store_location")
    .agg(total_revenue=("revenue", "sum"))
    .reset_index()
)

plt.figure(figsize=(8,5))

sns.barplot(
    data=store_revenue,
    x="store_location",
    y="total_revenue"
)

plt.title("Revenue by Store Location")
plt.xlabel("Store Location")
plt.ylabel("Revenue")
plt.xticks(rotation=15)
plt.show()
```

